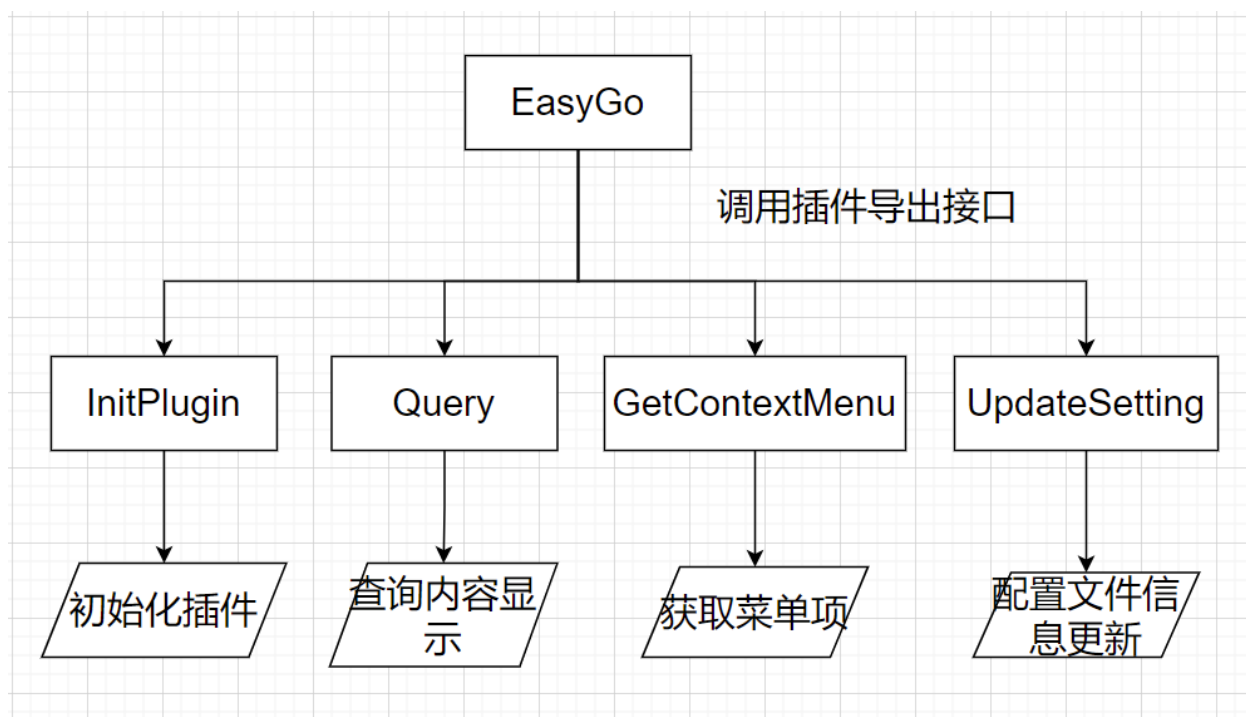


简介

EasyGo借鉴了wox的插件架构，尤其是python插件，几乎差不多，所以如果你开发过wox的python插件，那么可以很容易的将它移植到EasyGo上面来

架构

插件的整体架构如图



插件接口介绍

插件导出三个基本接口函数供EasyGo调用，大概介绍如下：

```
1 #ifdef CMDPLUGIN_EXPORTS
2 #define CMDPLUGIN_API extern "C" __declspec(dllexport)
3 #else
4 #define CMDPLUGIN_API extern "C" __declspec(dllimport)
5 #endif
6
7 CMDPLUGIN_API bool InitPlugin(char* pPluginPath);
8 CMDPLUGIN_API bool Query(char* pQuery, char* pResult, int* length);
9 CMDPLUGIN_API bool GetContextMenu(char* result, char* pMenu, int* length);
10 CMDPLUGIN_API void UpdateSetting();
11
12 /**
13  * 初始化插件，插件可以在这里做一些准备性工作，比如保存传入的插件路径，用来创建配置
```

```

14  * [in] pPluginPath: 当前插件位于的文件夹路径
15  */
16  bool InitPlugin(char* pPluginPath)
17  {
18      return true;
19  }
20
21  /**
22  * 查询函数，使用EasyGo传入的查询关键字来获取结果列表并返回
23  * [in] pQuery: 用户输入的查询文字，格式为json字符串
24  * [out] pResult: 用户返回的结果数组，格式为json字符串
25  * [in_out] length: pResult的空间大小
26  * EasyGo会进行两次调用，第一次调用，pResult = NULL，我们需要返回结果的长度，然后EasyGo
27  * 足够大的空间，然后传入pResult指针来获取结果，使用完毕后记得释放pResult的内存
28  */
29  bool Query(char* pQuery, char* pResult, int* length)
30  {
31      return true;
32  }
33
34  /**
35  * 获取针对某条结果的菜单选项，一般用不上，我们返回false即可
36  * [in] result: 结果列表中的某一条结果，格式为json字符串
37  * [out] pMenu: 返回的菜单数组，格式为json字符串
38  * [in_out] length: pMenu的空间大小，同Query函数，会调用两次
39  *
40  */
41  bool GetContextMenu(char* result, char* pMenu, int* length)
42  {
43      return true;
44  }
45
46  // 这个是当用户修改了配置文件，通知插件更新，不需要可以忽略
47  void UpdateSetting()
48  {
49  }

```

数据结构介绍

接着我们来说明一下这三个函数涉及到的json字符串结构

```
bool Query(char* pQuery, char* pResult, int* length)
```

参数: pQuery

```
1 {  
2     "RawQuery": "lm xxxx",  
3     "Keyword": "lm",  
4     "Parameter": "xxxx"  
5 }
```

它可以转化为下面的结构体

```
1 struct QueryText{  
2     string rawQuery;  
3     string keyword;  
4     string parameter;  
5 };
```

参数: pResult

```
1 {  
2     "Results": [  
3         {  
4             "Title": "结果1",  
5             "ShowType": 0,  
6             "SubTitle": "这是结果1",  
7             "IconPath": "app.png",  
8             "Action": {  
9                 "FuncName": "Ra_CopyPath",  
10                "Parameter": "参数",  
11                "HideWindow": true  
12            }  
13        },  
14        {  
15            "Title": "结果2",  
16            "ShowType": 0,  
17            "SubTitle": "这是结果2",  
18            "IconPath": "app.png",  
19            "Action": {  
20                "FuncName": "Ra_CopyPath",  
21                "Parameter": "参数",  
22            }  
23        }  
24    ]  
25 }
```

```

22         "HideWindow": true
23     }
24 }
25 ]
26 }

```

它可以转化成如下的结构体

```

1 struct PluginAction{
2     string funcName;           //响应的函数名，如果是自定义函数，需要将此函数导出，
3     string parameter;         //传入函数的参数，如果你有许多参数，请使用json字符串
4     bool hideWindow;          //用户回车或者单击完成操作后是否隐藏EasyGo输入窗口
5
6     PluginAction(){
7         hideWindow = true;
8     }
9 };
10
11 //这个是Result的完整结构体，Query和GetContextMenu函数的一些参数都只是用了其中部分
12 struct Result{
13     string id;                 //这个是插件id，由EasyGo赋值
14     int showType;              //这个是显示方式，默认是0，如果设置为1，则会读取SubTitle的文
15     string title;              //这是EasyGo上行文字
16     string subTitle;           //这是EasyGo下行文字
17     string iconPath;           //这是图标路径，如果为空那么默认使用插件图标
18     string extraData;          //这是额外数据，如果你想在GetContextMenu中获取一些自定义
19     PluginAction action;       // 这个是Result的响应函数定义，定义用户回车或者鼠标左
20 };

```

```
bool GetContextMenu(char* result, char* pMenu, int* length)
```

参数: result

```

1 {
2     "ID": "6064551FD7904BD09D23FB9CA9F4CD03",
3     "Title": "结果1",
4     "SubTitle": "这是结果1",
5     "ExtraData": "这个是可选的，由Query中赋值得来"
6 }

```

参数: pMenu

同Query函数的pResult, 是json数组

EasyGo提供给插件的接口

EasyGo提供给插件的有两种类型接口, 主动的和被动的, 大部分是被动调用, 涉及到PluginAction 结构体。

PluginAction里面有个FuncName, 在EasyGo里面定义了一些常用的函数, 方便用户调用, 名字和作用如下:

Ra_CopyPath : 会拷贝Parameter文字内容到剪切板

Ra_Copy : 这个会拷贝Parameter指定的文件到剪切板, 你可以在桌面上进行粘贴文件操作

Ra_CopyImage : 拷贝图片到剪切板, 可以进行图片粘贴操作

Ra_ChangeQuery : 会设置EasyGo输入框的值为Parameter字符串, 如果和之前输入框的文字不同会触发一次查询操作

Ra_Delete : 删除Paramter参数指定的文件或者文件夹 (不可恢复, 非移动到回收站, 请注意)

Ra_Recycle : 删除Paramter参数指定的文件或者文件夹到回收站

Ra_Open : 打开Parameter指定的可执行程序或者文件夹

Ra_OpenFileFolder : 打开Parameter指定文件路径所在的文件夹

Ra_OpenWeb : 打开Parameter指定的网址

Ra_ShowMsg : 会弹出消息提示框, 非阻塞

Ra_ShowTip : 会在系统右下角弹出提示框

Ra_ShowContent : 会弹出一个文本浏览框显示你传入的Parameter内容, 支持文本和图片, 只读, 图片格式如下:

"[pic=图片路径]" : 字符串, 支持本地路径和网络路径

Ra_EditFile : 会弹出一个文本框, 显示你传入的Parameter路径文件的内容, 纯文本, 支持编辑

Ra_PlayMusic : 播放本地或者网络音乐, 传入音乐地址, 再次调用则会暂停音乐

Ra_PauseMusic : 暂停当前播放的音乐

Ra_StopMusic : 停止当前播放的音乐

以上是被动调用的，根据返回的结果来进行用户点击的响应设置

然后是主动的，主动的目前只提供了如下函数，可以直接使用GetProcAddress来获取此导出函数的函数指针进行调用，注意调用约定为 stdcall

`void Ra_ChangeQuery(const char* strQuery)` : 修改输入框的查询字符串,query为字符串类型，如果文本发生改变会触发重新查询

`void Ra_ReQuery()` : 强制重新查询

`void Ra_Reload()` : 重载

`void Ra_ShowMsg(const char* title,const char* msg)` : 显示消息框，title为标题，msg是消息内容

`void Ra_ShowTip(const char* title , const char* msg)` : 会在系统右下角弹出提示框，title为标题，msg是消息内容

`void Ra_ShowContent(const char* title,const char* msg)` : 会弹出一个文本浏览框显示你传入的Parameter内容，支持图文，只读，图片格式如下：

`"[pic=图片路径]"` : 字符串，支持本地路径和网络路径

`void Ra_EditFile(const char* title,const char* filePath)` : 会弹出一个文本框，显示你传入的文件的内容，纯文本，支持编辑

`void Ra_PlayMusic(const char* musicPath)` : 播放本地或者网络音乐，传入音乐地址，再次调用则会暂停音乐

`void Ra_PauseMusic()` : 暂停当前播放的音乐

`void Ra_StopMusic()` : 停止当前播放的音乐

配置文件

```
1 {
2     "ID":"98451A7B5C764F6A9A3814EF7E4AFEEC",           这个是guid唯一标示
3     "Keyword":"guid",                                   这个是插件关键字，用户使用该默认关键字出发插件
4     "Argc":1,                                           这个是代表插件参数个数，关键字+子关键字
5     "Name":"生成Guid",                                插件名
6     "Description":"生成Guid",                          插件描述
7     "Author":"zsxeee",                                  插件作者
8     "Version":"1.0.0",                                  插件版本
9     "PluginType":"python",                              插件类型，目前有c++，python和e三种
10    "PluginMode":"EnterMode",                           插件模式，EnterMode或者RealMode，此字段可以有
11    "AcceptType": "png,ico,folder",                      插件支持的拖入的文件类型，就是文件的后缀名，
12    "EnableSeparate": 1,                                  是否开启插件分离窗口的支持，默认为1开启
```

```
13  "IconPath": "img\\app.png",           插件图标，请使用相对路径
14  "ExeName": "Main.py" ,               插件源文件名字
15  "CfgPath": "set.json"               插件的配置选项，使用相对路径，指定json配置文件路径。可
16 }
```

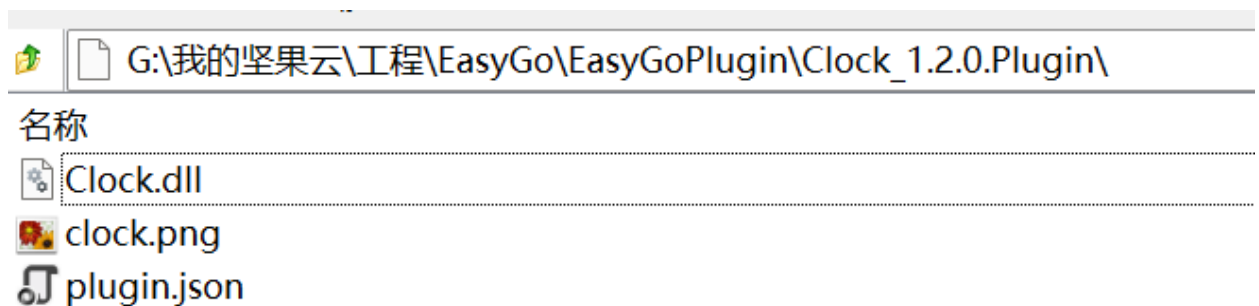
插件CfgPath配置文件

配置文件json，配置项均为文本格式，而且不支持多级，类似map结构

```
1 {
2     "name": "admin",
3     "password": "admin",
4     "role": "admin"
5 }
```

插件打包

插件打包后的格式是 **插件名.Plugin**，其实就是将插件的三个文件打包成zip文件，然后把zip后缀改成plugin即可，记住了，打包的时候插件的外层不要有文件夹



如果你有说明文件，那么请加入 **readme.txt** 文件，全小写，然后加入帮助内容，用户在插件管理页面可以通过右键详情打开帮助文档，然后你也可以放入手动配置文件，在plugin.json指定（见上），然后用户可以在插件管理页面通过右键设置打开配置文件进行配置

具体的使用方法请参考github源码示例